new value, only one will be first, and the second will therefore fail. One possibility is to combine add and change into a single member function, as you did for the DNS cache in listing 3.13.

The only other interesting point from an interface perspective is the *if any* part of obtaining an associated value. One option is to allow the user to provide a "default" result that's returned in the case when the key isn't present:

```
mapped_type get_value(key_type const& key, mapped_type
default_value);
```

In this case, a default-constructed instance of `mapped_type` could be used if the `default_value` wasn't explicitly provided. This could also be extended to return a `std::pair<mapped_type,bool>` instead of just an instance of `mapped_type`, where the `bool` indicates whether the value was present. Another option is to return a smart pointer referring to the value; if the pointer value is `NULL`, there was no value to return.

As already mentioned, once the interface has been decided, then (assuming no interface race conditions) the thread safety could be guaranteed by using a single mutex and a simple lock around every member function to protect the underlying data structure. However, this would squander the possibilities for concurrency provided by the separate functions for reading the data structure and modifying it. One option is to use a mutex that supports multiple reader threads or a single writer thread, such as the `boost::shared_mutex` used in listing 3.13. Although this would indeed improve the possibilities for concurrent access, only one thread could modify the data structure at a time. Ideally, you'd like to do better than that.

#### Designing a Map Data Structure for Fine-Grained Locking

As with the queue discussed in section 6.2.3, in order to permit fine-grained locking you need to look carefully at the details of the data structure rather than just wrapping a preexisting container such as `std::map<>`. There are